

Technical Interview & Assessment Question Bank

Curated Comprehensive Questions on Large Language Models, Production RAG, and Agentic AI Architectures

REFINED & STANDARDIZED VERSION

FOUNDATIONS OF LLMS & RAG

Question 1: What are Large Language Models (LLMs), and how do Retrieval-Augmented Generation (RAG) frameworks complement their inherent capabilities and limitations?

Question 2: An external knowledge base is often used in RAG systems to keep data up-to-date. Can you explain how the retrieval process works within a RAG framework and how it integrates with the LLM during the text generation phase?

PRODUCTION PIPELINES & SCALABILITY

Question 3: When building production-grade RAG solutions, what specific engineering strategies and best practices ensure the scalability and security of the underlying data pipelines?

Question 4: Decoupled ingestion and vector database optimization are critical for scalability. How do you handle vector store updates in real-time scenarios without impacting query latency or search performance?

DATA PROCESSING & CHUNKING STRATEGIES

Question 5: In document processing and embedding pipelines, what are the key steps involved in executing efficient document ingestion, metadata extraction, and pre-processing?

Question 6: Given the constraints of LLM context windows, could you analyze different chunking strategies (e.g., fixed-size, semantic, recursive paragraph) and their direct impact on embedding quality and retrieval accuracy?

PROMPT ENGINEERING & ORCHESTRATION FRAMEWORKS

Question 7: Define prompt engineering and explain why it is critically important for optimizing and improving the deterministic quality of AI-generated responses.

Question 8: Prompt engineering helps minimize hallucinations and tightly controls response styles. What are the foundational strategies (such as few-shot prompting, chain-of-thought, or system instructions) used to achieve these structural outcomes?

Question 9: Setting a clear persona and applying explicit constraints are key prompt engineering practices. How does an orchestration framework like LangChain facilitate advanced retrieval processes to improve context relevance and output precision?

ADVANCED RETRIEVAL & VECTOR DATABASES

Question 10: In advanced retrieval strategies, concepts like context routing and intentional orchestration often surface. Can you clarify how intentional context routing and constraint-based control function in highly accurate enterprise AI architectures?

Question 11: Can you explain how chainless or modular control flows benefit AI retrieval workflows compared to traditional rigid linear pipelines?

Question 12: What are the core operational principles of vector retrieval workflows, and how do they systematically minimize factual errors and improve the grounding of AI outputs?

Question 13: Explain the underlying data structures and indexed-search principles behind vector databases (e.g., HNSW, IVF-PQ) and clarify how they fundamentally differ from traditional relational or text-search databases.

AGENTIC AI ARCHITECTURES

Question 14: What are the key architectural principles crucial for designing, optimizing, and monitoring agentic AI systems within complex, non-deterministic technical environments?

Question 15: Hierarchical multi-agent orchestration and state-machine-based execution guardrails are foundational to production agents. How do these concepts contribute to the systemic robustness, fault tolerance, and scalability of agentic systems?

Question 16: Describe the essential production frameworks, infrastructure components, or observation tools necessary to support the successful and reliable deployment of autonomous enterprise agents.

Question 17: What specific post-deployment optimization strategies (e.g., fine-tuning embeddings, prompt distillation, caching, or reinforcement learning from feedback) would you employ to maximize the latency-cost performance of an agentic system?

TECHNICAL CODING ASSESSMENT

Question 18: Coding Problem: You are given a list of integers. Write a highly efficient function that identifies and returns the *second largest unique number* in the list.

Requirements:

1. The function must correctly handle negative numbers and arbitrary duplicate values.
 2. If the list contains fewer than two unique numbers, the function must return `None`.
 3. The list will always contain at least one integer.
-